

Project Specification: Progr3SS Habit Planner & Tracker

This document outlines the functional specifications for the Progr3SS application

Splash Screen

- **Functionality:**
 - Displays a startup animation upon application launch.
 - Performs an automatic login check to authenticate the user.
 - If authentication is successful, the user is auto-logged in and redirected to the Home Screen.
 - If no valid authentication is found, or if auto-login fails, the user is redirected to the Login Screen.
- **Backend Interaction (Implicit):** - Token Refresh/Validation: the auto-login functionality involves validating an existing access-token or using a refresh-token to obtain new access-tokens. - Endpoint: POST /auth/local/refresh - Summary: Refresh access and refresh tokens. - Security: Requires refresh-token. - Responses: 200 with Tokens (accessToken, refreshToken).

Register Screen

- **Functionality:**
 - Allows a new user to register an account by entering a username, email, and a password, followed by a password confirmation.
 - **Password Mismatch Handling:** If the entered password and its confirmation do not match, the respective text fields will be highlighted with a red border, and hint text will provide details to the user.
 - **Successful Registration:** Upon successful registration, the user is immediately sent to the Home Screen.
 - **Optional:** Google Authentication: Users also have the option to register and authenticate using their Google account.
- **Backend Interaction:**
 - **Local Sign-Up:** - Endpoint: POST /auth/local/signup
 - Summary: Register user with email, password, and optional profile image.
 - Request Body (multipart/form-data): Requires username (string), email (string, format email), password (string, format password). Optional profileImage (string, format binary).
 - - Responses: 201 for successful registration, returning **AuthResponseDto** (message, user, tokens).
 - **Optional Google Sign-In/Registration:**
 - Endpoint: POST /auth/google
 - Summary: Authenticate user with Google ID token.
 - Request Body (application/json): Example includes idToken (string).
 - Responses: 200 for successful Google login, returning **AuthResponseDto** (message, user, tokens).

Optional Reset Password Screen

- **Functionality:**
 - Allows a user to reset their password by entering their registered email address.
 - After entering the email, the user waits for a message indicating that a new password has been generated and sent to their email.
- **Backend Interaction:**
 - **Reset Password via Email:**
 - Endpoint: POST /auth/reset-password-via-email
 - Summary: Reset user password and send via email.
 - Request Body (application/json): Requires email (string).
 - Responses: 200 indicating the new password was sent to the user's email.

Login Screen

- **Functionality:**
 - Users enter their email and password to log in.
 - The provided credentials are validated and sent to the backend for verification.
 - Upon successful login, the user is navigated to the main application interface (the Home Screen).
 - Provides a "Forgot password" option, which redirects to the Reset Password Screen.
- **Backend Interaction:**
 - **Local Sign-In:**
 - Endpoint: POST /auth/local/signin
 - Summary: Authenticate with email and password.
 - Request Body (application/json): Requires email (string) and password (string) as part of SignInDto.
 - Responses: 200 for successful login, returning AuthResponseDto (message, user, tokens).
- **Optional Google Login:**
 - Endpoint: POST /auth/google
 - Summary: Authenticate user with Google ID token.
 - Request Body (application/json): Example includes idToken (string).
 - Responses: 200 for successful Google login, returning AuthResponseDto (message, user, tokens).
- **Optional Forgot Password:** (See Reset Password Screen functionality above).
 - Endpoint: POST /auth/reset-password-via-email
 - Summary: Reset user password and send via email.

Home Screen

- **Functionality:**
 - Displays the user's schedule for the current day.
 - Schedules are sorted by the time of day.
 - Each scheduled item shows its current status (completed or not completed).
 - Provides a navigation option to go to the Add Schedule Screen.
- **Backend Interaction:**

- Get Schedules by Day:
- Endpoint: GET /schedule/day
- Summary: Get schedules by day (defaults to today).
- Parameters: Optional date (string, format YYYY-MM-DD, e.g., "2025-07-14") in query.
- Responses: 200 returning an array of ScheduleResponseDto for the given day. The ScheduleResponseDto includes status (Planned, Completed, Skipped) and start_time for sorting.
- Security: Requires access-token.

Create Schedule Screen

- Functionality:
 - Allows the user to either choose an existing defined habit or create a new habit.
 - Users can set a start time for the schedule.
 - Users can select a repeat pattern for the schedule (e.g., daily, weekdays, weekends).
 - Users can define a goal duration for the habit within the schedule.
 - **Optional** Partners: Users can add other participants to the schedule.
 - Options to Create (save) or Cancel (discard) the schedule.
- Backend Interaction:
 - List Habits:
 - Endpoint: GET /habit
 - Summary: List all habits.
 - Responses: 200 returning an array of HabitResponseDto.
 - Security: Requires access-token.
 - Create a New Habit: (See Add Habit functionality below).
 - Endpoint: POST /habit
 - Summary: Create a new habit.
 - Create Custom Schedule (Manual Time):
 - Endpoint: POST /schedule/custom
 - Summary: Create a custom schedule.
 - Request Body (application/json): Requires habitId (number), date (date-time string), start_time (date-time string), is_custom (boolean, default true). Optional fields include end_time (date-time string), duration_minutes (number), participantIds (array of numbers), notes (string).
 - Responses: 201 for custom schedule created, returning ScheduleResponseDto.
 - Security: Requires access-token.
 - Create Recurring Schedule:
 - Endpoint: POST /schedule/recurring
 - Summary: Create a recurring schedule (e.g., weekdays, weekends).
 - Request Body (application/json): Requires habitId (number), start_time (date-time string), repeatPattern (string, enum: "none", "daily", "weekdays", "weekends", default "none"), is_custom (boolean, default true). Optional fields include end_time (date-time string), duration_minutes (number), repeatDays (number, default 30), participantIds (array of numbers), notes (string).
 - Responses: 201 for recurring schedules created, returning an array of ScheduleResponseDto.
 - Security: Requires access-token.

- Create Weekday Recurring Schedule:
 - Endpoint: POST /schedule/recurring/weekdays
 - Summary: Create schedules for specific weekdays.
 - Request Body (application/json): Requires habitId (number), start_time (date-time string), daysOfWeek (array of numbers, 1=Monday ... 7=Sunday), numberOfWeeks (number, default 4). Optional fields include duration_minutes (number), end_time (date-time string), participantIds (array of numbers), notes (string).
 - Responses: 201 for schedules created, returning an array of ScheduleResponseDto.
 - Security: Requires access-token.

Add Habit

- Functionality:
 - Allows the user to define a new habit by providing:
 - A Habit Name.
 - A Description (a short motivation or explanation behind the habit).
 - A Goal for the habit.
 - A Category selection, choosing an icon to define the habit's category.
- Backend Interaction:
 - Create a New Habit:
 - Endpoint: POST /habit
 - Summary: Create a new habit.
 - Request Body (application/json): Requires name (string, e.g., "Morning Run"), categoryId (number, e.g., 1), goal (string, e.g., "Run 10 times in 2 weeks"). Optional description (string, e.g., "Run 2km every morning").
 - Responses: 201 for creation, returning HabitResponseDto.
 - Security: Requires access-token.
 - Get Habit Categories:
 - Endpoint: GET /habit/categories
 - Summary: Get habit categories.
 - Responses: 200 returning an array of HabitCategoryResponseDto (id, name, iconUrl).
 - Security: Requires access-token.

Add Progress

- Functionality:
 - Enables users to update their progress for a specific schedule.
 - Users can add optional notes related to the progress update.
 - Progress updates contribute towards the overall completion status of a schedule.
- Backend Interaction:
 - Create Progress for a Schedule:
 - Endpoint: POST /progress
 - Summary: Create progress for a schedule.
 - Request Body (application/json): Requires scheduleId (number) and date (string). Optional logged_time (number), notes (string), is_completed (boolean).
 - Responses: 201 for progress creation, returning ProgressResponseDto.
 - Security: Requires access-token.

Schedule Details Screen

- Functionality:
 - Displays comprehensive details of a specific habit associated with the schedule.
 - Includes a progress bar to visualize completion.
 - Shows notes related to the habits or schedule.
 - Provides access to recent activities, showing historical data related to the schedule.
- Backend Interaction:
 - Get Schedule by ID:
 - Endpoint: GET /schedule/{id}
 - Summary: Get schedule by ID.
 - Parameters: id (number, required) in path.
 - Responses: 200 returning ScheduleResponseDto. This DTO contains:
 - habit: Details of the associated habit (HabitResponseDto).
 - notes: Optional notes for the schedule.
 - progress: An array of ProgressResponseDto representing historical progress for the schedule, including date, logged_time, notes, and is_completed.
 - Security: Requires access-token.

Edit Schedule Screen

- Functionality:
 - Allows modification of the schedule's Start Time and End Time.
 - Users can modify the total duration of the schedule.
 - Ability to set the schedule's status to 'Planned', 'Skipped', or 'Completed'.
 - Users can add or remove participants/partners.
 - Provides functionality to edit or update notes associated with the schedule.
- Backend Interaction:
 - Update Schedule by ID:
 - Endpoint: PATCH /schedule/{id}
 - Summary: Update schedule by ID.
 - Parameters: id (number, required) in path.
 - Request Body (application/json): UpdateScheduleDto allows modifying: start_time, end_time, duration_minutes, status (enum: "Planned", "Completed", "Skipped"), date, is_custom, participantIds (array of numbers), notes (string).
 - Responses: 200 for schedule updated, returning ScheduleResponseDto.
 - Security: Requires access-token.

Edit Notes

- Functionality:
 - Enables users to edit notes directly from the Schedule Details Screen.
 - Users can enter an edit mode to make changes.
 - A save action is required to preserve the updated note.
 - This feature is designed for easier note management.
- Backend Interaction:
 - This functionality is part of the Update Schedule by ID API.

- Endpoint: PATCH /schedule/{id}
- Summary: Update schedule by ID.
- Request Body (application/json): The UpdateScheduleDto includes a notes field (string) that can be updated.
- Security: Requires access-token.

Delete Schedule

- Functionality:
 - A delete button is accessible via an options menu.
 - Upon selection, a popup window appears, asking for confirmation ("Are you sure?") before proceeding with the deletion.
- Backend Interaction:
 - Delete Schedule by ID:
 - Endpoint: DELETE /schedule/{id}
 - Summary: Delete schedule by ID.
 - Parameters: id (number, required) in path.
 - Responses: 204 for schedule deleted successfully.
 - Security: Requires access-token.

Profile Screen

- Functionality:
 - Displays a summary of the user's profile data.
 - Allows the user to check their habits and progress.
 - Provides an option to add new habits.
 - Includes a logout option with a confirmation dialog.
- Backend Interaction:
 - Get Current User Profile:
 - Endpoint: GET /profile
 - Summary: Get My Profile.
 - Responses: 200 returning ProfileResponseDto (id, email, username, description, profileImageUrl, etc.).
 - Security: Requires access-token.
 - Check Habits by User ID:
 - Endpoint: GET /habit/user/{userId}
 - Summary: Find habits by user ID.
 - Parameters: userId (number, required) in path.
 - Responses: 200 returning an array of HabitResponseDto.
 - Security: Requires access-token.
 - Add New Habits: (See Add Habit functionality).
 - Endpoint: POST /habit
 - Summary: Create a new habit.
 - Logout:
 - Endpoint: POST /auth/local/logout
 - Summary: Logs out the user.
 - Responses: 200 for successfully logged out.

- Security: Requires access-token.

Edit Profile Screen

- Functionality:
 - Accessible from the Profile Screen.
 - Allows the user to set a new username.
 - Displays the user's email address and profile photo.
- Backend Interaction:
 - Update My Profile:
 - Endpoint: PATCH /profile
 - Summary: Update My Profile.
 - Request Body (application/json): UpdateProfileDto. Can be used to update the username.
 - Responses: 200 for profile updated, returning ProfileResponseDto.
 - Security: Requires access-token.
 - Upload Profile Image:
 - Endpoint: POST /profile/upload-profile-image
 - Summary: Upload profile image.
 - Request Body (multipart/form-data): Requires profileImage (string, format binary) as part of UpdateProfileImageDto.
 - Responses: 200 for profile image uploaded successfully, returning ProfileResponseDto.
 - Security: Requires access-token.

Optional AI Assistant Screen

- Functionality:
 - Integrates with the OpenAI API.
 - Enables users to ask for habit suggestions and health tips.
 - Provides options to accept or decline given schedule recommendations.